Java 入门笔记

第十课 · Java 方法与可变参数

学习目标 掌握方法的定义、调用、参数、返回值和局部变量作用域，并正确使用可变参数。

目录

本课目标 2

5.1 什么是方法 2

5.2 方法定义 2

5.3 方法调用 3

5.4 返回值与 return 3

5.5 参数与局部变量 4

5.6 方法设计原则 4

5.7 可变参数 4

5.8 综合示例：统计任意数量整数 5

5.9 命令行参数与方法配合 6

常见错误 6

本课小结 6

自测题 6

整理日期：2026 年 11 月 1 日

> 数组把一组数据集中保存，方法把一段功能集中定义。将重复逻辑封装为方法，可以减少重复代码，让程序结构更清晰。

本课目标

- 掌握方法的定义、调用、参数和返回值。
- 区分形式参数、实际参数、void 与有返回值的方法。
- 理解局部变量作用域和方法调用过程。
- 掌握可变参数的语法、规则及其数组本质。

5.1 什么是方法

方法是一段具有名称、可以被调用的代码。它通常接收输入，执行处理，并选择是否返回结果。

```
调用者准备实参
    ↓
进入被调用方法并建立局部变量
    ↓
执行方法体
    ↓
return 返回结果或方法正常结束
    ↓
回到调用位置继续执行
```

5.2 方法定义

```
修饰符 返回值类型 方法名(参数列表) {
   方法体;
   return 返回值;
}
```

示例：

```
public static int add(int a, int b) {
   int result = a + b;
   return result;
}
```

组成部分	示例	含义
修饰符	public static	可见性与调用方式
返回值类型	int	方法返回一个整数
方法名	add	调用时使用的标识符
参数列表	int a, int b	方法需要的输入
方法体	{ ... }	具体执行逻辑
return	return result;	结束方法并返回结果

方法定义在类体中，不能把一个普通方法定义在另一个方法内部。

5.3 方法调用

```
int sum = add(10, 20);
System.out.println(sum);
```

定义中的 a、b 称为形式参数，调用时传入的 10、20 称为实际参数。实参的数量、顺序和类型必须与可调用的方法匹配。

5.3.1 完整示例

```
public class MethodDemo {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void printResult(int value) {
        System.out.println("结果：" + value);
    }

    public static void main(String[] args) {
        int result = add(10, 20);
        printResult(result);
    }
}
```

同一个类中的 static 方法可直接通过方法名互相调用。

5.4 返回值与 return

5.4.1 有返回值的方法

```
public static double average(double[] values) {
    double sum = 0;
    for (double value : values) {
        sum += value;
    }
    return sum / values.length;
}
```

返回表达式的类型必须能赋给声明的返回值类型。执行 return 后，方法立即结束。

5.4.2 void 方法

void 表示不返回结果值。

```
public static void printLine() {
    System.out.println("----------");
}
```

void 方法可以使用单独的 return; 提前结束。

```
public static void printPositive(int number) {
    if (number <= 0) {
        return;
    }
    System.out.println(number);
}
```

5.5 参数与局部变量

参数和方法体中声明的变量都是局部变量，只在本次方法调用范围内有效。

```
public static int square(int number) {
    int result = number * number;
    return result;
}
```

方法调用结束后，本次调用的局部变量不再可用。不同调用拥有各自独立的参数和局部变量。

5.6 方法设计原则

- 一个方法尽量完成一个明确任务。
- 方法名使用动词或动词短语，例如 calculateAverage、printMenu。
- 需要给调用者继续使用的结果应通过 return 返回，而不是只在方法内部输出。
- 对不合法输入进行检查，例如空数组、除数为 0。
- 避免依赖含义不清的全局状态。

5.7 可变参数

5.7.1 语法

当实参数量不确定但类型相同时，可以使用可变参数。

```
public static int sum(int... numbers) {
    int result = 0;
    for (int number : numbers) {
        result += number;
    }
    return result;
}
```

调用时可以传入任意数量的 int：

```
sum();
sum(10, 20);
sum(1, 2, 3, 4, 5);
```

在方法体中，numbers 按 int[] 使用，因此可访问 length、下标并进行遍历。

5.7.2 可变参数规则

- 可变参数必须位于参数列表最右侧。
- 一个方法最多只能声明一个可变参数。
- 调用时可以传入零个、一个或多个对应类型的实参。
- 也可以直接传入现有数组。

```
public static void printScores(String name, int... scores) {
    System.out.println(name + " 的成绩：");
    for (int score : scores) {
        System.out.println(score);
    }
}
```

错误示例：

```
public static void wrong(int... numbers, String name) { } // 可变参数不在末尾
```

5.8 综合示例：统计任意数量整数

```
public class VariableArgumentsDemo {
    public static int sum(int... numbers) {
        int result = 0;
        for (int number : numbers) {
            result += number;
        }
        return result;
    }

    public static double average(int... numbers) {
        if (numbers.length == 0) {
            throw new IllegalArgumentException("至少需要一个数字");
        }
        return (double) sum(numbers) / numbers.length;
    }

    public static void main(String[] args) {
        System.out.println("总和：" + sum(10, 20, 30));
        System.out.printf("平均值：%.2f%n", average(10, 20, 30));

        int[] values = {2, 4, 6, 8};
        System.out.println("数组总和：" + sum(values));
    }
}
```

average 需要至少一个数字，所以先检查数组长度。可变参数只是一种调用语法，方法内部仍然要处理空数组等边界情况。

5.9 命令行参数与方法配合

```
public class ArgumentProduct {
    public static int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("用法：java ArgumentProduct 整数1 整数2");
            return;
        }

        try {
            int first = Integer.parseInt(args[0]);
            int second = Integer.parseInt(args[1]);
            System.out.println("乘积：" + multiply(first, second));
        } catch (NumberFormatException exception) {
            System.out.println("参数必须是整数");
        }
    }
}
```

main 负责接收和检查外部输入，multiply 负责计算。这种职责分离便于阅读、复用和测试。

常见错误

- 只定义方法却从未调用，误以为它会自动执行。
- 有返回值的方法缺少必要的 return。
- void 方法试图返回一个值。
- 实参数量、顺序或类型与形参不匹配，或把普通方法定义在 main 内部。
- 可变参数未放在参数列表最后，或声明了多个可变参数。
- 对零个可变参数直接计算平均值，造成除以 0。

本课小结

- 方法通过“参数输入 - 方法体处理 - 返回值输出”组织功能。
- void 方法不返回结果值，有返回值的方法必须返回兼容类型。
- 参数和局部变量只在当前调用范围内有效；可变参数在方法体中就是数组。
- 第十一课将在方法基础上进入类和对象，并解释参数传递和方法重载。

自测题

1. 形式参数和实际参数有什么区别？
2. return; 与 return value; 分别适用于什么方法？
3. 为什么可变参数必须在参数列表最右侧？
4. 编写 max(int... numbers)，返回最大值并处理没有参数的情况。